

Tarea Unidad 9

Estación IoT inteligente con n8n, ESP32 y LLM

Automatización de procesos y agentes IA

Programación para IA

CE en Inteligencia Artificial y Big Data

Antonio Ferrer Martínez

Abril 2026

Rev 2.0

Índice de contenidos

1. Introducción

Esta tarea plantea diseñar y montar un workflow en n8n que resuelva un problema real o, al menos, realista. En clase vimos un ejemplo con RAG sobre Google Drive y un chatbot que respondía a preguntas sobre documentos indexados. Para esta entrega cada uno tenía que proponer y construir su propio flujo con al menos un componente de IA y alguna integración útil. Una indicación explícita del enunciado era no replicar el RAG visto en clase salvo que se ampliara lo suficiente, cosa que no hago: mi proyecto va por otro lado.

Mi propuesta es una estación IoT inteligente montada sobre un ESP32 con varios sensores del kit Elegoo (temperatura, humedad, presencia, distancia y luz). Los sensores envían sus lecturas por HTTP a un workflow de n8n que combina los datos del interior con la información meteorológica real del exterior (Open-Meteo), pasa el conjunto al modelo LLM de Groq para generar un análisis en lenguaje natural, avisa por Telegram, guarda el histórico en Google Sheets y, como devuelve la respuesta al propio ESP32, la placa actualiza su LED RGB y su buzzer.

Sobre esa base he añadido dos mejoras que suben nota: un chatbot de Telegram que usa embeddings de Gemini para contestar preguntas sobre el histórico, y una predicción diaria que se dispara cada mañana a las 8:00 combinando el pronóstico de Open-Meteo con el LLM. También hay auto-localización por IP para que el usuario no tenga que configurar la ubicación manualmente.

En mitad del desarrollo el servidor de n8n del profesor se cayó, así que migré todo a una instalación self-hosted con Docker usando el kit oficial de n8n (self-hosted-ai-starter-kit), y expongo el n8n local a Internet con ngrok para que Telegram pueda llegar al webhook. Al final resulta que esa migración me ha dejado una infraestructura más robusta y además es una mejora más que documentar.

2. Objetivos

El enunciado fija unos requisitos mínimos y algunos opcionales que suman nota. Repaso cada uno y explico cómo lo cumplo:

2.1 Requisitos obligatorios

- **Uso de IA:** el workflow usa el modelo **llama-3.1-8b-instant** de Groq. Ese modelo recibe el estado de los sensores interiores junto con los datos del exterior y devuelve un análisis en español con un nivel de alerta (VERDE, AMARILLO o ROJO).
- **Fuente de datos real:** los datos vienen de dos sitios reales, no inventados. Los interiores los genera el ESP32 con sus sensores físicos. Los exteriores los consulto a Open-Meteo, un servicio meteorológico gratuito con datos reales.
- **Salida útil:** mensaje a Telegram con el análisis de la IA, fila en Google Sheets como histórico y respuesta al propio ESP32 con el nivel de alerta para que actualice el LED RGB y el buzzer.
- **Más de 5 nodos:** el workflow tiene 32 nodos distribuidos en cinco ramas (flujo principal, geolocalización web, chatbot con embeddings, predicción diaria y manejador de errores).
- **Gestión de errores:** hay un nodo Error Trigger que captura cualquier fallo y manda un aviso por Telegram con el nodo que falló. Varios nodos Code también hacen validaciones defensivas sobre los datos antes de seguir.
- **No es un RAG:** el proyecto no es una copia del ejemplo de clase. No indexa documentos en vector store para responder preguntas sobre ellos. La parte de embeddings (chatbot) es una mejora adicional sobre un flujo IoT, no el núcleo del sistema.

2.2 Requisitos opcionales que cumplo

- **Embeddings:** el chatbot de Telegram calcula el embedding de la pregunta con **gemini-embedding-001** de Google y lo pasa junto con el histórico al LLM para generar la respuesta.
- **Varios servicios externos:** ESP32 por HTTP, Open-Meteo API, Groq LLM API, Gemini embeddings API, Telegram Bot API, Google Sheets vía Apps Script, Nominatim para geocoding y ip-api.com para localización por IP. Ocho servicios distintos conectados.
- **Doble canal de geolocalización:** una mini web con la API nativa del navegador, ubicación compartida vía Telegram y auto-localización por IP como fallback.
- **Hardware IoT real:** la estación se monta sobre un ESP32 con siete componentes del kit Elegoo (sensores y actuadores), no es un simulador.
- **Feedback físico:** el ESP32 recibe el nivel de alerta como respuesta HTTP y enciende el LED RGB (verde, amarillo, rojo) y activa el buzzer en nivel rojo.
- **Predicción diaria:** schedule trigger que a las 8:00 cada mañana combina el forecast de 24 horas de Open-Meteo con el LLM y envía por Telegram un resumen del tiempo previsto.

3. Arquitectura general

La arquitectura tiene tres piezas principales y varios servicios colgando a los lados. Lo resumo primero en un párrafo y luego lo desgloso.

El ESP32 lee sus sensores cada 30 segundos y envía los datos por HTTP POST a un webhook de n8n. El workflow valida los datos, resuelve la ubicación (por prioridad: staticData manual, IP geolocation, o sin ubicación), consulta Open-Meteo con la lat/lon, pasa todo al modelo LLM de Groq, parsea su respuesta, manda un mensaje a Telegram, escribe una fila en Google Sheets y, por último, responde al ESP32 con el nivel de alerta. En paralelo corren otras tres ramas: el chatbot con embeddings, la predicción diaria y el error handler.

3.1 Capas

Capa de sensores (hardware): ESP32 DevKit V1 con DHT11 (temperatura y humedad), HC-SR501 (presencia por PIR), fotorresistencia LDR en divisor de tensión con una resistencia de 10 kilohmios, ultrasónico HC-SR04 (distancia), OLED 0.96" I2C, LED RGB de cátodo común con tres resistencias de 220 ohmios y un buzzer activo.

Capa de orquestación: workflow de n8n con 32 nodos. Hace de cerebro del sistema. Recibe los datos, decide qué hacer con ellos, llama al LLM, al servicio de embeddings, manda avisos y responde al ESP32.

Capa de IA: Groq (llama-3.1-8b-instant) para generación de texto, Gemini (gemini-embedding-001) para embeddings vectoriales. Los dos son servicios gratuitos en tier free.

Capa de infraestructura: n8n self-hosted en Docker corriendo en mi PC con el kit oficial self-hosted-ai-starter-kit, expuesto a Internet con un túnel ngrok para que Telegram pueda alcanzar los webhooks. El Apps Script de Google Sheets y el resto de servicios son en la nube.

3.2 Flujo de datos de una lectura

Este es el recorrido que hace una lectura de sensores desde que sale del ESP32 hasta que vuelve a él con el veredicto de la IA:

Paso	Componente	Acción
1	ESP32	Lee DHT11, LDR, PIR y HC-SR04. Construye un JSON con temp, hum, luz, presencia, distancia.
2	ESP32	HTTP POST al webhook de n8n (ngrok) con los datos.
3	n8n - Webhook	Recibe el POST y pasa el body al siguiente nodo.
4	n8n - Validar	Comprueba campos obligatorios, valida rangos, extrae la IP pública del header X-Forwarded-For.
5	n8n - IP Geolocation	Si no hay ubicación en staticData, consulta ip-api.com con la IP pública y la guarda.
6	n8n - Merge	Decide qué ubicación usar (manual > IP > ninguna).
7	n8n - IF	Bifurca según si hay ubicación configurada.
8	n8n - Open-Meteo	Consulta el tiempo actual en esa lat/lon.
9	n8n - Combinar	Une interior + exterior, calcula diferencias, detecta anomalías.
10	n8n - Groq	Llama al LLM con el prompt construido a partir de los datos.
11	n8n - Parsear IA	Extrae el texto del LLM, detecta el nivel de alerta y prepara el

Paso	Componente	Acción
		mensaje.
12	n8n - Salidas	Envía el mensaje a Telegram y escribe una fila en Google Sheets en paralelo.
13	n8n - Responder	Devuelve un JSON al ESP32 con nivel y ciudad.
14	ESP32	Recibe el JSON, actualiza el LED RGB y, si es nivel ROJO, dispara el buzzer.

4. Entorno n8n: del servidor del profesor a Docker + ngrok

La primera versión del proyecto corría sobre la instancia de n8n del profesor en progian8n.duckdns.org. Pero a mitad del desarrollo esa instancia dejó de responder, así que moví todo a mi propio n8n self-hosted con Docker. A efectos prácticos resultó una mejora: tengo control total sobre el workflow, puedo hacer snapshots, reiniciar cuando quiera y no dependo de que el servidor esté arriba.

4.1 Instalación del self-hosted-ai-starter-kit

El equipo de n8n mantiene un kit oficial llamado self-hosted-ai-starter-kit que levanta con Docker Compose toda una pila local pensada para IA:

- n8n en el puerto 5678.
- Ollama, un runtime de modelos LLM locales (no lo uso en este proyecto, pero está disponible por si quiero probar sin depender de Groq).
- Qdrant, base de datos vectorial (tampoco la uso aquí, pero viene incluida).
- PostgreSQL para el almacenamiento interno de n8n.

El arranque fue clonar el repo, ajustar las variables de entorno en `docker-compose.yml` y lanzar con:

```
docker compose up -d
```

4.2 Exposición a Internet con ngrok

El problema de un n8n local es que los webhooks no son accesibles desde Internet. Telegram no puede mandar mensajes a `localhost:5678`, y la mini web tampoco puede POSTear ahí si la abro en el móvil fuera de la red de casa. Para eso uso ngrok: registro gratuito, authtoken, y un solo comando levanta un túnel HTTPS que redirige a mi n8n local.

```
ngrok http 5678
```

Me devuelve una URL pública tipo `https://tapping-widget-entering.ngrok-free.dev` que redirige a `http://localhost:5678`. Con esa URL configuro el webhook de Telegram (con la API del bot) y actualizo las URLs que uso en el ESP32 y la mini web.

4.3 Variables de entorno clave

Para que n8n genere las URLs de webhook con la URL pública de ngrok en vez de localhost, añadí tres variables al servicio n8n en `docker-compose.yml`:

```
- WEBHOOK_URL=https://tapping-widget-entering.ngrok-free.dev - N8N_HOST=tapping-widget-entering.ngrok-free.dev - N8N_PROTOCOL=https
```

Después de modificar el fichero, un `docker compose down` y `up` para que cojan las nuevas variables. Ya entonces al importar el workflow los paths de los webhooks aparecen con la URL pública correcta.

5. Workflow n8n detallado

El workflow tiene 32 nodos distribuidos en cinco ramas. Las describo por bloques para no hacer la lectura pesada.

5.1 Rama principal (sensores → IA → salidas)

Es el recorrido central: llega una lectura del ESP32 y sale una respuesta con análisis IA. 13 nodos.

- **1. Webhook Sensores ESP32:** punto de entrada para las lecturas. Método POST en la ruta /webhook/estacion-iot. responseMode en "responseNode" para poder devolver al final del flujo el nivel de alerta al ESP32.
- **2. Validar Datos Sensor (Code):** comprueba los campos obligatorios (temp, hum, luz, presencia), valida rangos, extrae la IP pública del header X-Forwarded-For y lee la ubicación actual del staticData del workflow.
- **2a. IP Geolocation (HTTP):** llama a ip-api.com con la IP pública del ESP32. Tiene "Continue on fail" activo para que no rompa el flujo si la IP es privada o el servicio no responde.
- **2b. Merge IP + staticData (Code):** si ya había una ubicación manual en staticData, la mantiene. Si no, y la llamada a ip-api funcionó, usa esas coordenadas y las persiste en staticData para futuras ejecuciones. Esta es la auto-localización.
- **3. Tiene Ubicación? (IF):** bifurca entre la rama con ubicación (llamar a Open-Meteo) y la rama sin ubicación (solo datos del sensor).
- **4. Open-Meteo API (HTTP):** consulta el tiempo actual en esa lat/lon. Sin API key, es gratuito para uso personal.
- **5. Combinar Interior + Exterior (Code):** fusiona los dos bloques de datos, calcula diferencias de temperatura y humedad, y detecta anomalías con reglas simples (temp alta, humedad alta, diferencia interior/exterior grande, UV alto...).
- **5b. Sin Ubicación (Code):** versión reducida que se ejecuta cuando no hay ubicación. Aplica las reglas de anomalías que no necesitan datos exteriores.
- **6. Análisis IA (Groq LLM):** HTTP POST a la API de Groq con prompt system ("eres asistente IoT, responde en español conciso, termina con NIVEL: ...") y prompt user construido a partir de los datos. Modelo llama-3.1-8b-instant, temperature 0.7, max_tokens 250.
- **7. Parsear Respuesta IA (Code):** extrae el texto del LLM, detecta si contiene VERDE, AMARILLO o ROJO, y prepara el mensaje que se enviará a Telegram.
- **8. Enviar Telegram (HTTP):** POST a la Bot API de Telegram. Uso Form Urlencoded en vez de JSON porque el texto del LLM tiene saltos de línea y caracteres que rompían el escape del JSON body.
- **9. Google Sheets Log (HTTP):** POST a un Google Apps Script que yo mismo tengo desplegado como web app. Escribe una fila en la hoja. El Apps Script expone también un GET para leer el histórico, que lo usa el chatbot.
- **10. Responder al ESP32 (Respond to Webhook):** devuelve un JSON con nivel y ciudad. El ESP32 lo recibe como respuesta al POST y actualiza el LED RGB y el OLED.

5.2 Rama de geolocalización web

Tres nodos. Un webhook /estacion-iot-geo recibe lat/lon/ciudad desde la mini web HTML, un nodo Code guarda esos valores en staticData (el almacén persistente del workflow) y un nodo Respond to Webhook cierra la llamada con OK.

5.3 Rama chatbot Telegram con embeddings

Nueve nodos. Esta rama es la que cubre el requisito de embeddings y le añade una funcionalidad vistosa al proyecto: permite preguntar al bot cosas sobre el histórico.

- **CHAT 1. Webhook Chatbot:** endpoint al que Telegram envía los mensajes. Configuré el bot para que llame a este webhook usando la Bot API (setWebhook).
- **CHAT 2. Clasificar (Code):** decide qué tipo de mensaje es: ubicación compartida, comando (/start, /help) o texto libre. Las ubicaciones y comandos se resuelven en línea sin pasar por el LLM.
- **CHAT 3. Es pregunta? (IF):** bifurca entre el camino de chatbot (si es pregunta) y el de respuesta directa (ubicación/comando).
- **CHAT 4. Leer histórico Sheets (HTTP):** GET al endpoint del Apps Script con ?action=last&n=50 para traer las últimas 50 lecturas.
- **CHAT 5. Embedding Gemini (HTTP):** POST a la API de Gemini (generativelanguage.googleapis.com) con el modelo gemini-embedding-001 para calcular el vector semántico de la pregunta del usuario.
- **CHAT 6. Preparar prompt (Code):** formatea el histórico como texto tabulado y construye un prompt para el LLM. Incluye en los metadatos la dimensión del embedding calculado (solo como referencia de trazabilidad).
- **CHAT 7. Groq LLM (HTTP):** llama al LLM con el histórico como contexto y la pregunta. Temperatura más baja que en el flujo principal (0.5) para respuestas más factuales.
- **CHAT 8. Responder Telegram (HTTP):** manda la respuesta al usuario en Telegram.
- **CHAT 9. Responder directo:** rama para ubicaciones y comandos, responde con un mensaje precomputado por el nodo Clasificar.

Sobre los embeddings: en esta iteración el embedding de la pregunta se calcula pero el LLM razona sobre todas las filas del histórico tal cual. Es una versión simplificada. Una evolución natural sería también calcular embedding de cada fila, hacer similitud coseno y pasar al LLM solo las 3 o 5 filas más relevantes. No lo hago porque con 50 filas el LLM aguanta perfectamente el contexto y la precisión de la respuesta es buena.

5.4 Rama de predicción diaria

Seis nodos. Es el segundo requisito opcional fuerte que cubro.

- **FCST 1. Schedule diario 08:00:** trigger cron con expresión "0 8 * * *". Cada mañana a las 8:00 dispara la rama.
- **FCST 2. Ubicación real (Code):** recupera la ubicación actual del staticData. Si no hay ninguna, lanza un error (se captura por el error handler y manda aviso por Telegram pidiendo que se configure).
- **FCST 3. Forecast Open-Meteo 24h (HTTP):** consulta la previsión horaria para las próximas 24 horas en esa lat/lon: temperatura, humedad, probabilidad de precipitación e índice UV.
- **FCST 4. Preparar prompt (Code):** construye un prompt que pide al LLM una predicción corta en español empezando con el día y la ciudad, destacando máxima, mínima, lluvia y UV. Incluye una instrucción específica para que no confunda la ciudad con otra homónima de otro país (al principio me lo confundía con Cartagena de Colombia).
- **FCST 5. Groq LLM predicción (HTTP):** genera el texto de la predicción.

- **FCST 6. Enviar Telegram (HTTP):** manda la predicción al chat con la cabecera "PREDICCIÓN DIARIA - Ciudad".

5.5 Manejador de errores

Dos nodos. El Error Trigger se dispara cuando cualquier nodo lanza una excepción. En el body llega un objeto con el error y el nodo que falló. Un HTTP Request a Telegram avisa con "ERROR EN ESTACION IoT - Nodo: X - Mensaje: Y". Sin este avisador los errores se quedan en los logs y no te enteras hasta que revisas el panel de ejecuciones.

6. ESP32 y conexionado

6.1 Hardware y entorno de desarrollo

La placa es un ESP32 DevKit V1 (30 pines) del kit Elegoo, la misma que usé en la tarea 7. El entorno de desarrollo es PlatformIO en VS Code por coherencia con esa tarea, aunque dejo también una versión lista para Arduino IDE por si alguien prefiere ese flujo.

Componentes conectados:

- DHT11 - temperatura (0 a 50 °C) y humedad (20 a 90 %). Comunicación 1-Wire.
- HC-SR501 PIR - detector de movimiento por infrarrojos. Salida digital HIGH cuando detecta.
- HC-SR04 - sensor ultrasónico de distancia. Necesita alimentación a 5 V.
- LDR (fotorresistencia) en divisor de tensión con una R de 10 kΩ. ADC del ESP32.
- OLED SSD1306 0.96" por I2C, dirección 0x3C.
- LED RGB de cátodo común con tres resistencias de 220 Ω.
- Buzzer activo, suena con HIGH directo.

6.2 Conexionado

Tabla de conexionado. La columna "Función" explica para qué uso cada pin:

ESP32	Componente	Conexión / Función
GPIO 4	DHT11 DATA	GPIO 4 → pin S del DHT11. VCC a 3V3, GND a GND.
GPIO 15	HC-SR501 OUT	GPIO 15 → pin OUT del PIR. VCC a 3V3, GND a GND.
GPIO 34	LDR (ADC)	Divisor: LDR entre 3V3 y GPIO34, R 10 kΩ entre GPIO34 y GND.
GPIO 5	HC-SR04 TRIG	Pulso de 10 μs para medir.
GPIO 18	HC-SR04 ECHO	Ancho del pulso proporcional a la distancia.
5V (VIN)	HC-SR04 VCC	El HC-SR04 no funciona bien a 3.3 V, necesita 5 V.
GPIO 21	OLED SDA	I2C datos, pin por defecto.
GPIO 22	OLED SCL	I2C reloj, pin por defecto.
GPIO 2	LED R (+R 220)	GPIO 2 → R 220 → ánodo R → cátodo común → GND.
GPIO 16	LED G (+R 220)	GPIO 16 → R 220 → ánodo G → cátodo común → GND.
GPIO 17	LED B (+R 220)	GPIO 17 → R 220 → ánodo B → cátodo común → GND.
GPIO 19	Buzzer +	Pin + a GPIO 19, pin - a GND.
3V3	Alimentación	DHT11, PIR, LDR, OLED.
GND	Masa común	Todos los componentes.

Elegí los GPIO esquivando los que dan problemas de boot en el ESP32. GPIO 0 y GPIO 15 pueden afectar al arranque si están en HIGH, pero GPIO 15 como entrada con el PIR en reposo no da problema. El GPIO 2 tiene un LED integrado en la placa, así que cuando se enciende el rojo también parpadea el azul integrado.

6.3 Esquema visual

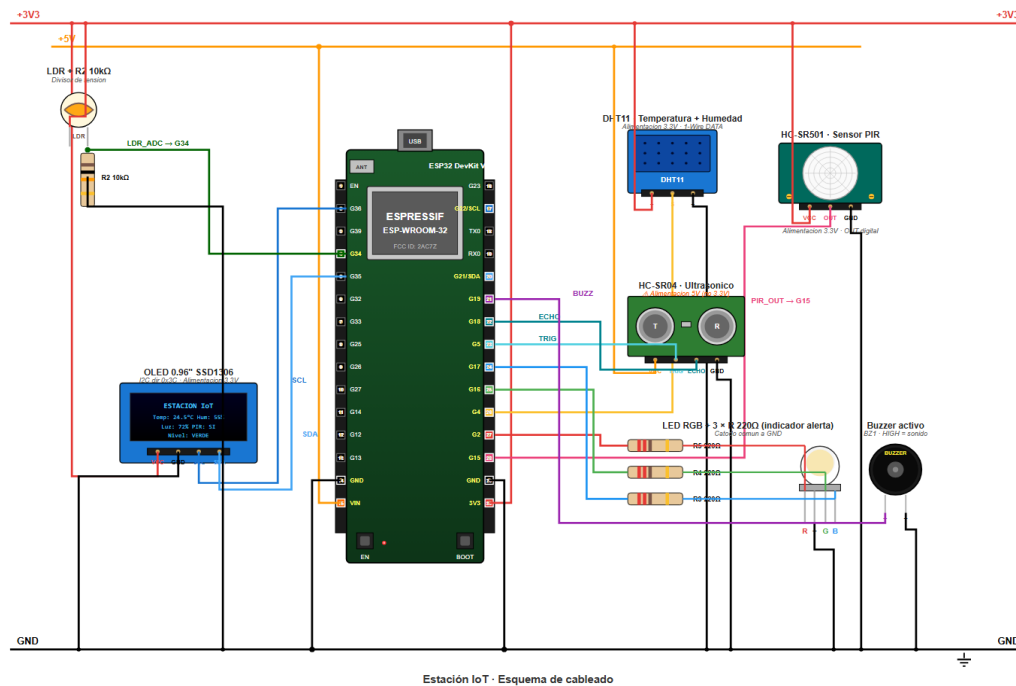


Figura 1. Esquema de cableado de la estación IoT.

6.4 Lógica del firmware

El bucle principal hace lo siguiente cada 30 segundos:

- Lee los cuatro sensores.
- Actualiza la pantalla OLED con los valores y la ciudad actual.
- Construye un JSON y lo envía por HTTP POST al webhook de n8n.
- Parsea la respuesta y actualiza el LED RGB (y el buzzer si el nivel es ROJO).

Entre lecturas el ESP32 atiende peticiones en un pequeño servidor HTTP propio en el puerto 80. Eso permite consultar el estado actual vía /estado o forzar un cambio de alerta vía /alerta desde la misma red local. Es útil como fallback si n8n no responde.

7. Geolocalización

El sistema necesita saber dónde está para pedir datos meteorológicos reales al exterior. Probé varias opciones y al final monté un sistema con tres niveles de prioridad:

7.1 Tres fuentes de ubicación

- **Nivel 1 - Configuración manual (preferida):** el usuario comparte su ubicación desde la mini web HTML o envía un mensaje de localización al bot de Telegram. El workflow guarda lat/lon/ciudad en el staticData.
- **Nivel 2 - Geolocalización por IP (auto, fallback):** si no hay ubicación manual configurada, el workflow extrae la IP pública del header X-Forwarded-For de la petición del ESP32, llama a ip-api.com con esa IP y obtiene una ubicación aproximada. Luego guarda esos valores en staticData para que las siguientes ejecuciones ya los tengan.
- **Nivel 3 - Sin ubicación (último recurso):** si ni la manual ni la IP están disponibles, el flujo salta la consulta a Open-Meteo y trabaja solo con datos interiores. La IA sigue analizando lo que hay.

7.2 Por qué este orden de prioridad

La IP geolocation es útil pero imprecisa: puede acertar la ciudad o desviarse hasta otra provincia según el ISP. Mi IP residencial de Movistar en Cartagena en las pruebas daba Barcelona una de cada cinco veces porque el ISP tiene infraestructura allí. Por eso la manual siempre gana. Probé también Google WiFi Geolocation pero requiere activar facturación en Google Cloud (aunque el tier gratuito cubre 40.000 peticiones al mes), así que lo descarté para este proyecto.

8. Google Sheets vía Apps Script

Para el histórico uso una hoja de Google Sheets llamada "Estación IoT" con 11 columnas. Lo lógico sería usar el nodo Google Sheets nativo de n8n, pero necesita OAuth de Google configurado en la instancia y no es trivial desde el Docker self-hosted.

La alternativa que uso es un Google Apps Script desplegado como web app en mi propia cuenta. Es un mini backend en JavaScript que corre en los servidores de Google y escribe en la hoja directamente, sin pasar por OAuth externo. El script expone dos endpoints:

- **doPost:** recibe un JSON desde n8n y añade una fila a la hoja. Limpia los posibles "=" que a veces n8n deja al principio de los valores (un bug que encontré depurando y que si no limpio hace que Sheets interprete los valores como fórmulas).
- **doGet:** expone tres acciones vía querystring. ?action=last&n=50 devuelve las últimas N filas, ?action=all las todas, ?action=stats genera un resumen. Lo usa el chatbot de la rama de embeddings para traer el histórico.

9. Análisis IA: Groq LLM y embeddings Gemini

9.1 Por qué Groq y llama-3.1-8b-instant

Para generación de texto necesitaba un modelo rápido, gratuito y compatible con el formato OpenAI (porque n8n se integra bien con esa interfaz). Probé tres opciones: Ollama local, Gemini y Groq.

- **Ollama local:** descartado porque requiere NGrok también y un modelo cargado en mi PC. Como el proyecto ya tiene suficientes capas, preferí una API externa.
- **Google Gemini:** tier gratuito generoso pero latencia algo mayor (800-1500 ms) y la API difiere ligeramente del estándar OpenAI.
- **Groq:** tier gratuito, compatibilidad OpenAI total, latencias muy bajas (100-300 ms) y varios modelos Llama disponibles. Este gana.

Dentro de Groq elegí llama-3.1-8b-instant porque es rápido, responde bien en español y es suficiente para la tarea de clasificar datos y generar una recomendación breve. Uso temperature 0.7 para que el texto salga natural y max_tokens 250 para acotar.

9.2 Por qué Gemini para embeddings

Groq no tiene endpoint de embeddings. Tenía que usar otro servicio. Opciones: OpenAI (de pago), Ollama (requería correr un modelo embedding adicional localmente) y Gemini (tier gratuito, API key sencilla de obtener desde aistudio.google.com).

El modelo elegido es gemini-embedding-001. Genera vectores de 768 dimensiones, rápido y suficiente para el tipo de preguntas cortas del chatbot. La API es un POST a generativelanguage.googleapis.com con la API key en el header X-goog-api-key.

Ahora mismo el embedding de la pregunta se calcula pero no lo uso para hacer similitud coseno en el código. Lo aprovecha el LLM como contexto junto con el histórico completo. Es una simplificación consciente: con 50 filas de histórico el LLM gestiona bien el contexto y la latencia sigue siendo baja. Una evolución natural sería guardar embeddings de cada fila en Qdrant (que ya tengo corriendo en Docker pero sin usar) y hacer similitud antes de pasar al LLM.

9.3 Prompts

El prompt del análisis principal:

```
Eres un asistente IoT. Analizas lecturas de sensores (interior) y datos meteo (exterior).  
Responde en español, max 3 frases. Termina con NIVEL: [VERDE/AMARILLO/ROJO].
```

El prompt del chatbot con embeddings:

```
Histórico de la estación IoT (N lecturas): [listado tabulado con fecha, ciudad, valores  
interior, valores exterior, alerta] Pregunta: <pregunta del usuario> Responde en español,  
conciso (max 4 frases), citando datos cuando sea relevante.
```

El prompt de la predicción diaria:

```
Eres un asistente IoT. Genera una predicción breve (max 4 frases) para hoy en <Ciudad>.  
Coordenadas reales: lat=..., lon=... Pronostico 24h (cada 3h): [listado por horas] Destaca:  
maxima/minima, lluvia, UV, recomendacion practica. Empieza con el día y la ciudad. NO  
inventes datos que no esten en el pronostico. NO confundas la ciudad con otra homonima en  
otro país.
```

La última instrucción ("no confundas con otra homónima") vino después de que el LLM me diera una predicción para Cartagena de Indias en Colombia en vez de Cartagena, Murcia. Añadí la aclaración explícita y ya no vuelve a pasar.

10. Instalación y puesta en marcha

10.1 Requisitos previos

- Cuenta de Telegram con un bot creado vía @BotFather (token del bot y chat_id propio).
- Cuenta de Groq con API key (console.groq.com).
- Cuenta de Google con API key de Gemini (aistudio.google.com/apikey).
- Cuenta de Google con una hoja de Google Sheets vacía llamada "Estación IoT".
- Docker Desktop instalado.
- Cuenta de ngrok gratuita (authtoken).
- ESP32 DevKit V1 con los componentes del kit Elegoo.
- VS Code con PlatformIO (o Arduino IDE) para compilar el firmware.

10.2 Pasos de instalación

1. Levantar n8n local con Docker

Clonar self-hosted-ai-starter-kit de GitHub, editar docker-compose.yml con las tres variables WEBHOOK_URL, N8N_HOST y N8N_PROTOCOL (apuntando a la URL de ngrok que obtendremos en el paso siguiente), y lanzar docker compose up -d.

2. Exponer n8n con ngrok

Instalar ngrok, configurar el authtoken con ngrok config add-authtoken <token>, y lanzar ngrok http 5678. Copiar la URL pública que devuelve.

3. Configurar Telegram

Crear bot con @BotFather, copiar token, mandarle un mensaje, y obtener el chat_id desde la API getUpdates. Configurar el webhook del bot con setWebhook apuntando a <url-ngrok>/webhook/estacion-iot-chatbot.

4. Desplegar el Apps Script

Abrir Google Sheets, Extensiones → Apps Script, pegar Code.gs, desplegar como web app con acceso "Cualquiera", copiar la URL.

5. Importar el workflow en n8n

Entrar en el n8n local en localhost:5678, crear workflow, pegar el JSON del workflow. Revisar los nodos Groq (API key), Gemini (API key), Telegram (token y chat_id) y Google Sheets (URL del Apps Script). Publish.

6. Compilar y subir el firmware al ESP32

Editar main.cpp con WIFI_SSID, WIFI_PASSWORD y N8N_WEBHOOK_URL. Compilar y subir con PlatformIO.

7. Configurar la ubicación

Opcionalmente, abrir la mini web geolocalizacion.html en el móvil y compartir la ubicación. Si no se hace, el sistema usará la IP pública del ESP32 para auto-localizar.

8. Verificar

Esperar 30 segundos después de encender el ESP32. Debería llegar un mensaje al Telegram con los datos y el análisis, y una fila nueva en Google Sheets.

11. Problemas encontrados y soluciones

Los listo en el orden en que aparecieron. Varios vinieron de cosas que no había encontrado en los tutoriales y tuve que depurar paso a paso.

11.1 El nodo Validar recibía los datos vacíos

El JSON del POST llegaba al webhook correctamente (se veía en la preview) pero el nodo Code inmediatamente después decía "campos faltantes: temp, hum, luz, presencia". Resulta que en esta versión de n8n el body a veces queda en \$input.first().json directamente y otras veces anidado en \$input.first().json.body. Se arregla con un OR:

```
const input = $input.first().json; const datos = input.body || input.data || input;
```

11.2 API key de Groq rechazada

Pegué la key, dio "Authorization failed". Al revisar vi que había copiado un carácter extra al principio. Regeneré una nueva, la pegué limpia y funcionó. No parece gran cosa pero perdí 15 minutos.

11.3 El nodo Parsear Respuesta IA fallaba por "nodo no ejecutado"

Intentaba leer datos de "5. Combinar Interior + Exterior", pero la ejecución había ido por la rama "5b. Sin Ubicación" porque no tenía ubicación configurada. Intentar acceder con \$() a un nodo no ejecutado lanza una excepción que no puedes atrapar con un simple try. Lo arreglé con try/catch anidados:

```
let datosAnteriores; try { datosAnteriores = $('5. Combinar Interior + Exterior').first().json; } catch(e) { try { datosAnteriores = $('5b. Sin Ubicacion (solo sensor)').first().json; } catch(e2) { datosAnteriores = {}; } }
```

11.4 Nodo Telegram: "JSON parameter needs to be valid JSON"

Al mandar el texto del LLM en el body JSON del HTTP Request, los saltos de línea, comillas y asteriscos rompían el parseo. Probé varias cosas (JSON.stringify en la expresión, fetch desde Code, \$http) pero ninguna estaba disponible en esta versión. La solución que me funciona es cambiar el body a Form Urlencoded y meter los parámetros como Body Fields. n8n se encarga del escape al serializar. Curiosamente la Bot API de Telegram acepta perfectamente form-urlencoded.

11.5 Google Sheets guardaba #ERROR! y #NAME?

Los primeros tests guardaban celdas con errores raros. Al clickar en la celda veía que el valor empezaba por "=" y Sheets lo interpretaba como fórmula. El motivo era que las expresiones de n8n con prefijo "=" a veces dejan ese carácter en el valor serializado. Lo arreglé en el Apps Script: antes de escribir cada valor, si empieza por "=", lo elimina. Así los textos se guardan como texto plano.

11.6 Google Sheets devolvía "connection aborted"

El nodo de lectura de Sheets fallaba con "connection aborted, perhaps the server is offline". Era porque Google Apps Script responde con un redirect 302 y el nodo HTTP Request por defecto no lo seguía. Lo arreglé activando "Follow All Redirects" en Options y subiendo el timeout a 30000 ms.

11.7 El modelo de embeddings de Google no existía

Empecé usando text-embedding-004 porque lo había leído en un tutorial, pero daba 404. Google había renombrado el modelo a gemini-embedding-001 y además requiere pasar

también el campo "model" dentro del body JSON (no solo en la URL). Con los dos cambios, la API respondió a la primera.

11.8 Cartagena de Indias en vez de Cartagena, Murcia

La predicción diaria empezó a hablar de Cartagena, Colombia. El LLM no tenía manera de saber a cuál de las dos me refería porque el prompt solo decía "Cartagena". Añadí en el nodo de ubicación que si la ciudad no tiene coma (ni país), le concatene ", España". Y añadí al prompt una instrucción explícita: "NO confundas la ciudad con otra homónima en otro país". Desde entonces siempre acierta.

11.9 El servidor de n8n del profesor se cayó

Mientras hacía las últimas pruebas, la instancia de progian8n.duckdns.org dejó de responder. Como ya había avanzado mucho, migré todo a mi propio n8n self-hosted con el kit oficial de n8n y expuse el webhook con ngrok. Tardé un buen rato (sobre todo con las variables de entorno para que n8n genere las URLs de webhook con la pública de ngrok en vez de localhost) pero al final el proyecto quedó más robusto: ya no depende del servidor del profesor.

11.10 staticData no persistía entre ejecuciones manuales y la schedule

Cuando metía una ubicación por web y luego lanzaba manualmente la rama de predicción desde el editor, el staticData aparecía vacío. Al principio pensé que había un bug de n8n pero por lo que leí es el comportamiento esperado: staticData solo persiste entre ejecuciones "productivas" (triggers reales), no desde el botón Execute del editor. En producción funciona porque la rama principal del ESP32 rellena staticData cada 30 segundos con la IP geolocation, y el schedule de las 8:00 usa el que quedó guardado.

12. Conclusiones

El proyecto cumple los requisitos del enunciado y añade tres mejoras que suben bastante la nota: chatbot con embeddings, predicción diaria y auto-localización por IP. Lo que más me ha aportado a nivel de aprendizaje es montar una cadena completa que une hardware real con servicios en la nube y un modelo LLM, respondiendo todo dentro del mismo bucle, y luego poder migrar toda esa infraestructura de un servidor externo a mi propio Docker sin romper nada.

Algunas reflexiones:

- n8n es potente pero tiene aristas que solo descubres cuando te topas con ellas (body.body, expresiones con "=" que se propagan, redirecciones HTTP no seguidas por defecto, staticData que depende del tipo de trigger). Una vez sabido, es rápido iterar.
- Groq es una alternativa muy atractiva a las APIs comerciales para prototipos. El tier gratuito es generoso y la latencia es mejor que Gemini o OpenAI.
- Cuando combinas varios servicios, el bug más probable no está en ninguno de ellos sino en el formato en que intercambian datos. He perdido más tiempo depurando problemas de serialización y redirecciones que escribiendo lógica propia.
- Tener desde el principio un manejador de errores que avisa por Telegram ha sido clave. Sin él, muchos fallos se habrían quedado enterrados en los logs del servidor.
- El kit self-hosted-ai-starter-kit es una puerta de entrada excelente al stack de n8n + Ollama + Qdrant. Aunque solo uso n8n de los tres, tener el resto ya levantado me abre camino a futuras iteraciones.

Lo siguiente que haría sería convertir el prompt del LLM en un agente con varias herramientas (consulta al histórico de Sheets para comparar con lecturas pasadas, consulta a la previsión meteorológica del día en lugar de solo al momento actual), usar Qdrant de verdad para los embeddings y añadir memoria conversacional al chatbot. Todo eso ya sería otra tarea.

13. Bibliografía y referencias

13.1 Documentación técnica

- n8n documentation - <https://docs.n8n.io/>
- self-hosted-ai-starter-kit - <https://github.com/n8n-io/self-hosted-ai-starter-kit>
- Groq API - <https://console.groq.com/docs/quickstart>
- Gemini embeddings - <https://ai.google.dev/gemini-api/docs/embeddings>
- Open-Meteo API - <https://open-meteo.com/en/docs>
- Telegram Bot API - <https://core.telegram.org/bots/api>
- ip-api.com - <https://ip-api.com/docs>
- Nominatim (OpenStreetMap) - <https://nominatim.org/release-docs/latest/api/Overview/>
- Google Apps Script web apps - <https://developers.google.com/apps-script/guides/web>
- ngrok documentation - <https://ngrok.com/docs>

13.2 ESP32 y sensores

- ESP32 Arduino Core - <https://docs.espressif.com/projects/arduino-esp32/>
- Adafruit DHT sensor library - <https://github.com/adafruit/DHT-sensor-library>
- Adafruit SSD1306 OLED - https://github.com/adafruit/Adafruit_SSD1306
- ArduinoJson - <https://arduinojson.org/>

13.3 Material de clase

- Apuntes de la Unidad 9 - Automatización de procesos y agentes IA (n8n_teoria.docx).
- Enunciado de la tarea (n8n_tarea_practica.docx).
- Código de la tarea 7 como referencia de estilo y arquitectura (control IoT de LEDs con MQTT).